

Serverless Architecture for Near Real-Time Processing of Biomedical Wearable Data in Non-Critical Hospital Areas

Lucía Revaliente Torres

Abstract— This work presents the design, implementation and evaluation of a distributed cloud architecture for the near real-time ingestion, processing, persistence and monitoring of biomedical data from wearable devices, using the continuous monitoring of hospital patients in non-critical areas as a use case. The architecture, deployed on AWS following a fully serverless model, integrates Amazon Kinesis Data Streams for ingestion, AWS Lambda for processing, Amazon S3 and DynamoDB for persistence, and Amazon CloudWatch for monitoring. The system has been evaluated through three experimental scenarios of increasing load (from ~ 8 ev/s to $\sim 4,304$ ev/s), defined from the real sampling frequencies of the WESAD dataset, with an end-to-end latency SLO of $P95 < 10$ s and a target data loss rate below 1%. Results confirm that the serverless paradigm is a viable architectural foundation for continuous biomedical monitoring in non-critical areas: the base and sustained scenarios meet all SLOs (maximum $P95$ of 1,074 ms, 0.00% data loss), while the peak scenario reveals the undersizing of the ingestion layer as the limiting factor. The experimental diagnosis identifies Kinesis throttling as the main cause and demonstrates that the entire measured delay accumulates in the Kinesis buffer and not in the internal Lambda process, guiding future optimizations to the ingestion layer and the design of the partition keys.

Index Terms—Serverless architectures, near real-time processing, biomedical monitoring, wearable devices, cloud computing, streaming data ingestion, AWS, Kinesis, Lambda.

Resumen— Este trabajo presenta el diseño, la implementación y la evaluación de una arquitectura *cloud* distribuida para la ingestión, el procesamiento, la persistencia y la monitorización casi en tiempo real de datos biomédicos provenientes de dispositivos *wearable*, considerando como caso de uso la monitorización continua de pacientes hospitalarios en áreas no críticas. La arquitectura, desplegada en AWS mediante un modelo completamente *serverless*, integra Amazon Kinesis Data Streams para la ingestión, AWS Lambda para el procesamiento, Amazon S3 y DynamoDB para la persistencia y Amazon CloudWatch para la monitorización. El sistema se ha evaluado mediante tres escenarios de carga creciente (de ~ 8 ev/s a ~ 4.304 ev/s), definidos a partir de las frecuencias reales del *dataset* WESAD, con un SLE de $P95 < 10$ s y una tasa de pérdida inferior al 1%. Los resultados confirman que el paradigma *serverless* es una base arquitectónica viable para la monitorización biomédica continua en áreas no críticas: los escenarios base y sostenido cumplen la totalidad de los SLOs ($P95$ máximo 1.074 ms, pérdida 0,00%), mientras que el escenario de pico revela el subdimensionamiento de la capa. El diagnóstico experimental identifica el *throttling* de Kinesis como causa principal y demuestra que la totalidad del retraso medido se acumula en el *buffer* de Kinesis y no en el procesamiento interno de Lambda, orientando así las optimizaciones futuras hacia la capa de ingestión y el diseño de las *partition keys*.

Palabras clave—Arquitecturas *serverless*, procesamiento en casi tiempo real, monitorización biomédica, dispositivos *wearable*, computación en la nube, ingestión de datos en *streaming*, AWS, Kinesis, Lambda.

1 INTRODUCTION

In hospital settings, particularly in non-critical areas, patient monitoring is typically based on vital-sign measurements taken at set intervals. This model of discontinuous follow-up can hinder the early detection of changes in patients' physiological state, meaning that certain alterations may go unnoticed or be detected with delay [1]. Faced with this scenario, wearable devices have emerged as an alternative that enables progress towards continuous monitoring models, generating continuous data streams that describe the temporal evolution of multiple physiological variables [2][3][4].

However, the availability of such data poses new technological challenges. In scenarios with multiple patients monitored simultaneously, systems must handle continuous streams of heterogeneous data, process large volumes

of information with low latency, and adapt to load increases without compromising the availability or integrity of the information [5][6][7]. In this context, cloud computing provides a suitable foundation for addressing these requirements, offering managed services for streaming ingestion, distributed processing and scalable storage [6][7]. Within this paradigm, serverless architectures allow operational complexity to be reduced and facilitate the dynamic adaptation of resources to workload [8][9].

Building on this motivation, this work aims to experimentally demonstrate whether the serverless model is a viable architectural foundation for the continuous monitoring of hospital patients in non-critical areas. To achieve this, the system is evaluated under conditions of increasing load derived from real wearable-device frequencies, analysing its behaviour in terms of latency, throughput, integrity, scalability and operational cost. The results make it possible to identify the limits of the current configuration and the limiting factors under extreme load.

• Contact e-mail: lucia.revaliente@autonoma.cat
• Work supervised by: Esteve Alquezar Mora (Computer Architecture and Technology Area)
• Academic year 2025/26

The report is organised into nine chapters, from the state of the art and the architecture design through to the experimental evaluation and conclusions.

2 STATE OF ART

2.1 Wearable-based biomedical monitoring

The literature shows growing interest in the use of wearable devices for the continuous monitoring of patients, both in clinical and home settings. Several studies have explored their application in cardiovascular monitoring, the early detection of physiological alterations, and support for chronic-disease management [2][3][4]. These works highlight the need for infrastructures capable of efficiently processing the continuous data streams generated [2][5].

2.2 Cloud architectures for biomedical data processing

To address this challenge, several works have proposed cloud-computing-based architectures. Bahmani et al. [6] and Orro et al. [7] propose platforms for the management and monitoring of health data, highlighting the advantages in scalability and availability, but also the need to control latency, integrity and operational supervision.

2.3 Serverless architectures

In this context, serverless architectures have emerged as an alternative for deploying distributed near real-time processing systems. This paradigm delegates infrastructure management to the cloud provider and automatically adapts resources to load. Recent studies highlight their scalability and operational simplicity, but also point out limitations in latency, observability and provider dependency [8][9][10].

2.4 Positioning of the work

The reviewed literature highlights the potential of wearable devices, cloud platforms and serverless architectures for biomedical monitoring. However, the behaviour of these systems under increasing load in near real-time scenarios has been little studied. This work contributes to addressing this gap through the design, implementation and experimental evaluation of a serverless architecture on AWS, across three load scenarios derived from real wearable-device frequencies.

3 OBJECTIVES AND METHODOLOGY

3.1 General objective

The objective of this work is to design, implement and experimentally evaluate a distributed cloud architecture based on serverless services for the near real-time processing of biomedical data from wearable devices, analysing its behaviour in terms of latency, throughput, integrity, scalability and operational cost. The ultimate goal is to determine the viability of this model as an architectural foundation for the continuous monitoring of hospital patients in non-critical areas.

3.2 Specific objectives

To achieve this general objective, the following specific objectives are set out:

- Identify the requirements of a near real-time biomedical data processing system.
- Design a distributed architecture based on managed cloud services.
- Implement a functional prototype deployed on AWS.
- Define an experimental methodology to evaluate the system's behaviour.
- Experimentally analyse the system's behaviour in terms of latency, throughput, integrity and scalability, identify the main bottlenecks, and determine the limits of the configuration under extreme load.

3.3 Scope

The scope of the work includes the ingestion, processing, persistence and monitoring of biomedical data, as well as the experimental evaluation of the proposed architecture. Excluded from the project's scope are the clinical validation of the results, the use of real devices in hospital settings, the application of machine learning (ML) models to the processed data, and the development of user interfaces for healthcare staff or patients.

3.4. Methodology

The work follows an iterative methodology. This approach allows the architectural decisions made to be progressively analysed and improvements to be introduced to the prototype as limitations or optimisation opportunities are detected.

In a first phase, the system requirements and the reference data model are established. Next, an architecture based on managed cloud services is designed. Subsequently, a functional prototype is implemented. Finally, the system is evaluated under different load levels to characterise its behaviour.

4 REQUIREMENTS AND DATA MODEL

4.1 System requirements

Based on the objectives defined previously, a set of non-functional requirements is established to guide both the architecture design and its subsequent experimental evaluation. Given that the system is oriented towards the continuous monitoring of patients in non-critical hospital areas, priority is given to the ability to process data with low latency, preserve its integrity, and adapt to load increases without compromising the overall functioning of the system. To evaluate these aspects, the requirements set out in Table 1 are defined.

Table 1. Non-functional requirements of the architecture.

Category	Requirement
Latency	Processing time below 10 seconds for P95
Integrity	Loss below 1% of processed events

Scalability	Ability to adapt to load increases without significant service degradation
Observability	Availability of metrics and logs to monitor system behaviour
Cost	Reduced operational cost through the use of managed cloud services

The main SLO is set at $P95 < 10,000$ ms, a threshold derived from the clinical context: in non-critical areas, relevant physiological alterations occur on timescales of minutes [1]. Therefore, a latency on the order of seconds does not compromise the clinical usefulness of the system. In addition, the P95 is used as a reference indicator to avoid occasional extreme values conditioning the evaluation [12].

4.2 Reference dataset

To evaluate the defined requirements, the WESAD (Wearable Stress and Affect Detection) dataset [13] is used as a reference. This dataset is widely used in research works related to the analysis of physiological signals. It contains multimodal data collected via wearable devices, including physiological and motion signals represented as continuous time series with heterogeneous sampling frequencies.

However, this data is not used as a direct source of events during the experiments. WESAD serves as a structural reference to define the data schema, the sampling frequencies and the multimodal nature of the records. Based on this information, a simulator capable of generating continuous streams of biomedical events is developed. Full details of the sensors and their frequencies are presented in Appendix 1.

4.3 Data model

The events generated by the simulator follow a common data model that defines the structure of the information throughout the whole pipeline. Given the heterogeneity of sampling frequencies between sensors, a record-per-sample approach is adopted: each sensor generates independent events sent according to its specific frequency. This makes it possible to represent multimodal data streams homogeneously and simplifies their distributed processing.

To preserve the traceability of the information, two levels of persistence are defined: raw data and processed data. Raw data corresponds to the original events generated by the simulator. Processed data, on the other hand, incorporates the temporal information and attributes generated during processing. This separation guarantees the reproducibility of results, facilitates the verification of integrity throughout the pipeline, and allows discrepancies between the original and processed data to be identified. The detailed field structure and record examples are presented in Appendix 1.

5 ARCHITECTURE DESIGN

5.1 Conceptual architecture

Based on the requirements defined previously, a distributed architecture is designed, based on an event-oriented processing model, structured in five main blocks: record generation, ingestion, processing, persistence and monitoring. Figure 1 presents a conceptual view of the system.

Each block assumes a specific responsibility within the pipeline, so that their separation facilitates the scalability of the system, simplifies the independent evolution of the components, and enables monitoring mechanisms to be incorporated without affecting the main processing flow.

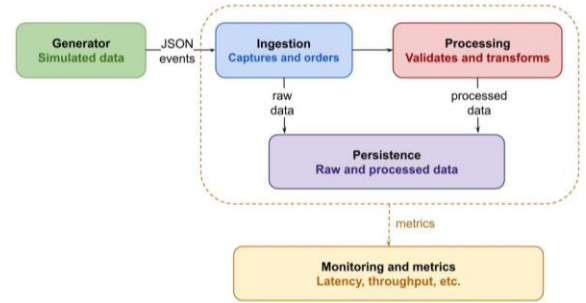


Figure 1. Conceptual model of the architecture and data flow.

5.2 AWS architecture

Based on the conceptual model, a fully serverless implementation based on Amazon Web Services (AWS) managed services is defined. AWS was chosen for the maturity of its streaming services, the native integration between serverless components, and the availability of a Free Tier. The resulting architecture structures the data flow into five blocks: generation, ingestion, processing, persistence and monitoring, as shown in Figure 2.

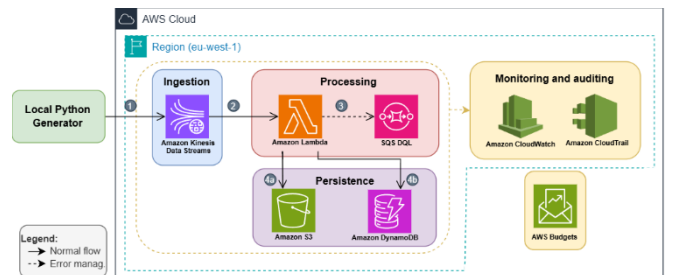


Figure 2. Diagram of the AWS architecture and data flow.

As can be seen, the biomedical events generated by the simulator are ingested via Kinesis Data Streams, processed by Lambda, and persisted in S3 (raw data) and DynamoDB (processed data). Failed events are sent to an SQS DLQ. In parallel, CloudWatch, CloudTrail and Budgets cover monitoring, auditing and cost control.

5.3 Architectural decisions

The architectural decisions adopted for each component of the system are detailed below. For each selected service, the main discarded alternative is included; full details are presented in Appendix 2.

5.3.1 Ingestion

Amazon Kinesis Data Streams was selected as the

ingestion service. In the context of this system, where multiple sensors generate continuous streams at heterogeneous frequencies (from 4 Hz to 700 Hz), Kinesis offers the necessary properties: continuous high-frequency streaming, preservation of temporal order per partition key (subject_id + sensor_type), spike management through buffering in the stream, and 24-hour retention for reprocessing in the event of failure [14]. Its native integration with Lambda via event source mapping allows ingestion to be decoupled from processing without additional configuration.

The main discarded alternative is Amazon MSK (Managed Streaming for Apache Kafka). Although it is the industry-standard reference for high-frequency streaming pipelines, the absence of a Free Tier, the disproportionate operational complexity for an individual prototype, and the need for additional configuration to integrate with Lambda made it unsuitable for this context. In a production scenario with hundreds of simultaneous patients, Kafka would be the preferred architectural decision.

5.3.2 Processing

AWS Lambda was selected as the main processing service. Lambda acts as the system's write point: it is responsible for persisting raw data to Amazon S3 and processed data to Amazon DynamoDB, consolidating the processing and writing operations into a single component [15]. For each event, it performs field validation, unit conversion, and enrichment with timestamps. Regarding performance, the cold-start phenomenon must be taken into account, which can introduce an additional one-off latency of 100–500 ms at the start of the scenarios, without compromising the SLO [15].

For error handling, Amazon SQS is added as a Dead Letter Queue (DLQ). Events that are not processed correctly are sent for later analysis.

The main discarded alternative is Apache Flink (Amazon Managed Service for Apache Flink). Although technically well suited for processing high-frequency streams, it introduces high operational complexity and a base cost without a Free Tier, which is incompatible with the prototype [16]. In a production scenario with complex transformation requirements or hundreds of simultaneous patients, Flink would be the chosen service.

5.3.3 Persistence

The persistence layer is organised into two tiers with differentiated services for each purpose, with Lambda as the system's sole writer.

Amazon S3 is used for raw data. Since raw records are kept for future reprocessing rather than for operational queries, the lack of structure poses no limitation: S3 allows events to be stored exactly as they arrive, with 99.99999999% durability and unlimited scalability [17].

Amazon DynamoDB is used for processed data. Monitoring the pipeline requires direct access to records by key (subject_id + event_id) with millisecond latency, a requirement that DynamoDB natively covers with a generous Free Tier [18].

The design of the partition keys is critical to avoid the hot-partition phenomenon, which occurs when too many

writes are concentrated in the same partition and can cause throttling. To mitigate this risk, the partition key is defined as the concatenation subject_id#sensor_type, distributing writes across as many partitions as there are active sensor types per subject. The event_id (UUID) is used as the sort key, guaranteeing the absolute uniqueness of each record and the idempotency of writes.

The main discarded alternative for processed data is S3+Athena: although more economical for occasional large-scale analysis, it introduces query latencies of between 2 and 30 seconds depending on volume and complexity, which are unsuitable for the operational monitoring of the pipeline.

5.3.4 Monitoring and auditing

Amazon CloudWatch was selected for collecting custom metrics from each block of the pipeline, with automatic alarms on key metrics and integrity verification by comparing ingested events against stored records [20]. AWS X-Ray was discarded due to the absence of a native alarm system and its inability to compare counts between blocks [21].

AWS CloudTrail is incorporated as a complementary auditing mechanism to record actions on the system's resources. Although the project does not address full compliance with clinical regulations, traceability of access and modifications is relevant in environments with potentially sensitive data. For operational cost control, AWS Budgets configures automatic alerts at 80–100% of the monthly budget of \$10.

5.4. Architectural trade-offs

The main architectural trade-offs of the system are presented below, analysing the cost and benefit of each design decision both in the context of the prototype and in its projection towards a production scenario.

5.4.1 Near real-time vs real-time

The adoption of a near real-time model, with latencies on the order of seconds, rather than a strictly real-time model, with latencies on the order of milliseconds, is the most decisive architectural decision of the system. It allows the use of serverless services such as Kinesis and Lambda at a much lower cost, and is sufficient for the use case considered, in which relevant physiological alterations occur on timescales of minutes [1][11]. This decision would remain valid in a production scenario in non-critical areas.

5.4.2 Serverless vs managed infrastructure

The fully serverless architecture maximises operational simplicity and minimises cost, but implies less control over low-level parameters: Lambda's internal parallelism, load distribution between shards, memory management per invocation, or behaviour under throttling conditions. In a production scenario with hundreds of simultaneous patients, the processing component would need to be reconsidered in favour of Apache Flink, which offers explicit control over parallelism, checkpoints and time windows, at the cost of higher operational complexity and a higher base cost.

5.4.3 Simplicity vs resilience in error handling

The prototype does not implement automatic reprocessing of DLQ events for simplicity. Failed events require manual intervention, an acceptable trade-off for the prototype but unacceptable in production. As a future evolution, automatic reprocessing could be implemented via a dedicated Lambda function or Step Functions.

5.4.4 Single-region vs multi-region

The prototype is deployed in a single region for reasons of cost and operational agility. In a real biomedical production environment, high availability and resilience would justify a multi-region architecture, especially given the sensitive nature of the data being managed. With infrastructure as code, the system could be relaunched in a second region within minutes.

6 IMPLEMENTATION

The implementation was developed following the Infrastructure as Code (IaC) paradigm using Terraform, defining the infrastructure declaratively and reproducibly [23]. All resources are managed through configuration files version-controlled in the project's public repository: <https://github.com/luciarevaliente/serverless-biomedical-nearrealtime-pipeline-aws>, where the README.md includes full implementation details and execution instructions.

Since the system does not use real physical devices, a simulator was developed that reproduces the behaviour of wearable devices, generating continuous streams of events towards Kinesis Data Streams. It is implemented as a Python script run locally, outside the AWS environment. For each sensor, it emits events at the corresponding sampling frequency using a `next_emit` mechanism that avoids temporal drift accumulation, ensuring that the ingestion rate remains stable throughout the duration of each scenario. In addition, it locally logs the events sent, facilitating subsequent validation of data integrity.

For each batch, the Lambda function writes the raw data to S3 as a single JSON object, reducing the number of PUT requests by a factor of 500 compared to per-event writing. It processes each event individually, adding latency timestamps, writes the processed records to DynamoDB, and sends failed events to the SQS DLQ. At the end of each invocation, it publishes custom metrics to CloudWatch under the BiomedicalPipeline namespace.

7 EXPERIMENTAL METHODOLOGY

This chapter describes the experimental framework designed to evaluate the behaviour of the architecture under different load levels, including the metrics, scenarios, configuration and limitations of the study.

7.1 Evaluation metrics

For each processed event, three timestamps are recorded within the AWS temporal domain: `kinesis_arrival_ts` is the moment at which Kinesis receives the event, generated by the service itself; `processing_start_ts` is the

moment at which Lambda begins individual processing of the event; and `processing_end_ts` is the moment at which validation and enrichment are completed. From these three timestamps, the metrics set out in Table 2 are calculated.

Table 2. System evaluation metrics.

Metric	Definition	Purpose
<code>kinesis_to_lambda_ms</code>	$= \text{processing_start_ts} - \text{kinesis_arrival_ts}$	Temps d'espera al buffer de Kinesis
<code>processing_latency_ms</code>	$= \text{processing_end_ts} - \text{processing_start_ts}$	Temps de processament de Lambda
<code>pipeline_latency_ms</code>	$= \text{processing_end_ts} - \text{kinesis_arrival_ts}$	Latència end-to-end dins del domini AWS. Mètrica principal SLO

The P95 is used as the reference indicator for the SLO [12], complemented by the P50 and P99 to characterise the full latency distribution and assess variability between repetitions.

7.2 Experimentals scenarios

Three scenarios of increasing load are defined, based on the real sampling frequencies of the WESAD dataset [13], as set out in Table 3.

Table 3. Experimental load-increase scenarios.

Scenario	Rate	Duration	SLO
Base (~8 ev/s)	1 subject; E4 (EDA + TEMP, 4 Hz)	5 min	P95 < 10 s; loss < 1%
Sustained (~208 ev/s)	2 subjects; E4 (BVP 64 Hz, ACC 32 Hz, EDA+TEMP 4 Hz)	5 min	P95 < 10 s; loss < 1%
Peak (~4,304 ev/s)	1 subject; E4 (4 sensors, 4-64 Hz) + RespiBAN (6 sensors, 700 Hz); ratio 175:1	30 s	P95 < 30 s; loss > 1%; backlog < 60 s

Note: the peak-scenario criteria are indicative. The primary objective is to quantify the degradation and identify the limiting factors.

The comparison between scenarios makes it possible to isolate two independent effects: the effect of the number of simultaneous subjects (base vs sustained) and the effect of extreme frequency heterogeneity (sustained vs peak).

Each scenario is run three consecutive times with a 60-second interval between repetitions. This interval is deliberate: in a real hospital environment, consecutive load spikes can occur without guaranteed recovery time. This allows the accumulation of backlog between repetitions to be observed as an experimental result in itself.

7.3 AWS service configuration

The service configuration was defined based on the experimental scenarios and the limits of each AWS service. Table 4 sets out the main parameters. The full configuration and justification are presented in Appendix 3.

Table 4. Configuration of the main AWS services.

Component	Configuration
Kinesis Data Stream	5 shards (PROVISIONED), retenció 24h
Lambda	Python 3.11, 128 MB
Lambda ESM	batch_size=500,

	parallelization_factor=10
DynamoDB	PAY_PER_REQUEST, batch_writer

The total cost of the complete experimental session is ~\$0.533, covering the three scenarios and the nine repetitions; the breakdown by service and an indicative production cost estimate are presented in Appendix 4.

7.4 Limitations of the experimental study

Several limitations must be taken into account when interpreting the results.

Regarding metrics, the generator-to-Kinesis segment is excluded from the main metric due to systematic clock skew between the local machine and the AWS domain. Therefore, `pipeline_latency_ms` represents the maximum latency measurable within the AWS domain, not the total absolute latency. In addition, `processing_latency_ms` excludes writes to S3 and DynamoDB, since `processing_end_ts` is recorded before these are executed. Incorporating them into the temporal model would require an additional write operation to DynamoDB per event, incompatible with the pipeline design. The CloudWatch invocation-duration data presented in Appendix 5 allow the impact of this to be estimated.

Regarding the environment, the experiments were run on a personal AWS account with no concurrent load, which may overestimate performance relative to a shared production environment. In addition, the simulator generates synthetic data that reproduces the WESAD dataset frequencies but not the temporal variability or artefacts specific to real physical devices.

Regarding experimental coverage, the evaluation covers a single final infrastructure configuration (5 shards, 128 MB Lambda), without studying the individual impact of each parameter. The iterative optimisation process of the AWS service configuration and the architecture implementation is detailed in Appendix 5. Finally, the scenarios do not include intermediate configurations that would allow the breaking point of the current configuration to be identified with greater precision.

8 EXPERIMENTAL EVALUATION

8.1 Results

Table 5 sets out the distribution of `pipeline_latency_ms` per scenario and repetition, allowing assessment of SLO compliance and reproducibility across runs. The breakdown of latency by pipeline component is presented in Appendix 6, and the ingestion and integrity data in Table 6.

Table 5. Distribution of `pipeline_latency_ms`.

Scenario	Run	P50 (ms)	P95 (ms)	P99 (ms)
Base (~8 ev/s)	1	564	1.037	1.063
	2	564	975	1.071
	3	565	1.038	1.115
Sustained (~208 ev/s)	1	608	1.074	1.698
	2	598	1.054	1.629
	3	597	1.060	1.632
Peak	1	49.911	134.184	150.538

(~4.304 ev/s)	2	75.394	156.835	165.952
	3	97.475	197.602	205.903

Taula 6. Dades d'ingestió per escenari i repetició.

Scenario	Run	Real rate (ev/s)	Events generated	Events accepted by Kinesis	Events stored	Total loss (%)
Base (~8 ev/s)	1-3	8,0	7.200	7.200	7.200	0,00%
Sustained (~208 ev/s)	1-3	208,0	187.198	187.198	187.198	0,00%
Peak (~4.304 ev/s)	1	2.782,5	124.302	113.931	112.132	9,79%
	2	3.533,5	108.535	91.654	90.054	17,03%
	3	3.884,0	120.470	104.046	100.970	16,18%

8.1.1 Base scenario

The base scenario establishes the performance baseline of the system under minimum-load conditions. In Table 5, the three repetitions show P95 values in the range 975–1,038 ms, well below the SLO of 10,000 ms. The experiments show high reproducibility: mean of 1,016.7 ms, standard deviation of 36.1 ms, and 95% CI of [927.0; 1,106.3] ms (t-distribution, n=3). In Table 6, the 7,200 events generated match exactly the 7,200 events stored in DynamoDB (0.000% loss).

Figure 3 shows the CDF of `pipeline_latency_ms` for the three repetitions. The three curves are presented almost overlapping, indicating a consistent distribution across runs.

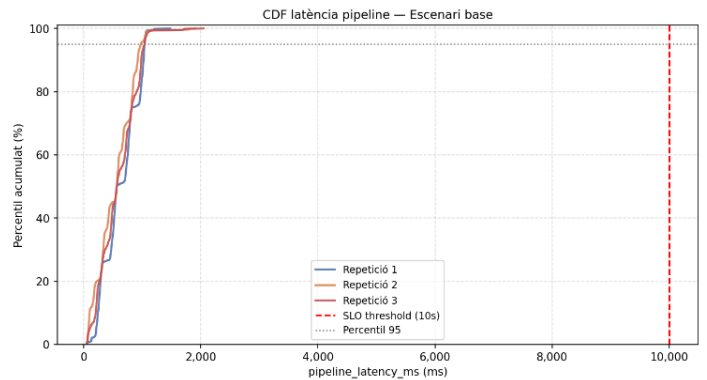


Figure 1. CDF of `pipeline_latency_ms` in the base scenario (~8 ev/s).

Regarding the AWS indicators, Kinesis's `IteratorAge`, which measures how long events remain in the buffer without being consumed, stays at 0 throughout the run, confirming that the pipeline consumes events with no accumulation. Lambda operates with a maximum of 3 concurrent invocations, consistent with the low ingestion rate. The DLQ records 0 messages and the error rate is 0%.

8.1.2 Sustained-load scenario

The sustained scenario evaluates the effect of

increasing the number of subjects and the heterogeneity of frequencies between sensors. In Table 5, the P95 values (1,074, 1,054 and 1,060 ms) show no monotonic growth between runs. The experiments show the highest reproducibility of the three scenarios: mean of 1,062.7 ms, standard deviation of 10.3 ms, and 95% CI of [1,037.2; 1,088.2] ms (t-distribution, $n=3$). In Table 6, the 187,198 events generated match exactly the 187,198 events stored (0.000% loss).

Figure 4 presents the P50, P95 and P99 bars per repetition, showing stable values across runs with no increasing trend.

Regarding the AWS indicators, Kinesis's IteratorAge stays at 0 throughout the run, confirming the absence of buffer accumulation. Lambda automatically scales from ≤ 3 to ≤ 11 concurrent invocations. The DLQ records 0 messages and the error rate is 0%.

8.1.3 Peak scenario

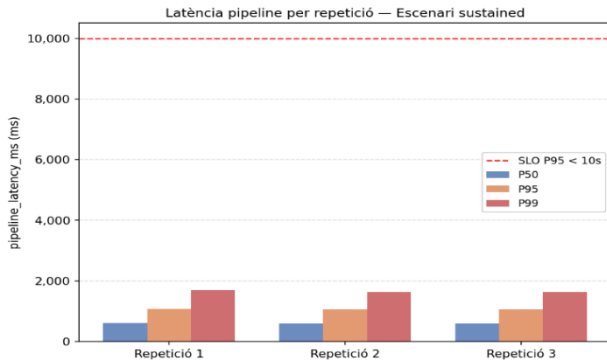


Figure 4. P50, P95 and P99 of `pipeline_latency_ms` per repetition in the sustained scenario (~ 208 ev/s).

The peak scenario evaluates the system's resilience under extreme load, exceeding the nominal capacity of the current configuration. In Table 5, the P95 grows monotonically between repetitions (134,184 ms, 156,835 ms and 197,602 ms), with high variability: mean of 162,873.7 ms, standard deviation of 32,137.4 ms, and 95% CI of [83,040; 242,707] ms (t-distribution, $n=3$). In Table 6, of the 353,307 events generated, 309,631 were accepted by Kinesis and 303,156 stored in DynamoDB, with a total loss of 14.195%. The DLQ records 0 messages and the Lambda error rate is 0%.

Figure 5 shows the temporal evolution of the P95 during the 30-second burst. Latency grows almost linearly from second 0, with no stabilisation phase.

Figure 6 presents the complete CDF of the three repetitions, in which the three distributions show a strong horizontal shift: Run 1 crosses the P95 at $\sim 134,184$ ms, Run 2 at $\sim 156,835$ ms, and Run 3 at $\sim 197,602$ ms.

Regarding the AWS indicators, Kinesis's IteratorAge grows progressively during the burst, reflecting backlog accumulation in the buffer. The post-burst backlog takes ~ 3.4 minutes to clear. Lambda operates with a maximum of 11 concurrent invocations, an error rate of 0%, and the DLQ records 0 messages.

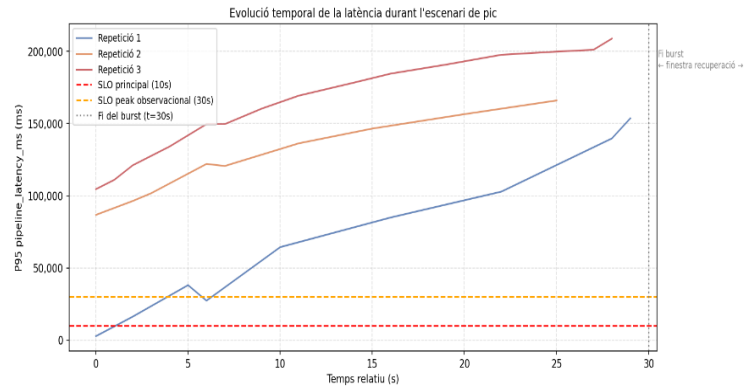


Figure 5. Temporal evolution of the P95 of `pipeline_latency_ms` during the peak scenario ($\sim 4,304$ ev/s).

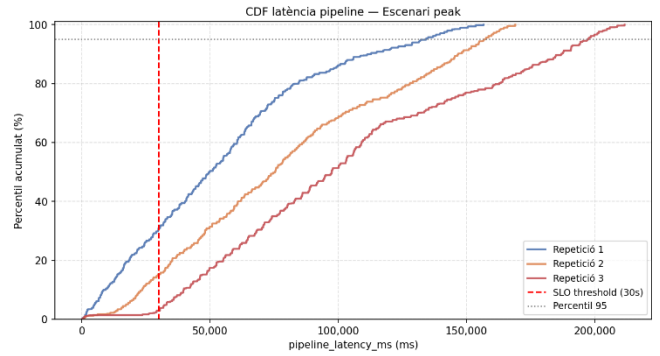


Figure 6. CDF of `pipeline_latency_ms` in the peak scenario ($\sim 4,304$ ev/s).

8.2 Analysis

The analysis is organised around three cross-cutting arguments that synthesise the experimental results, independently of the specific scenario in which they are observed.

8.2.1 The serverless architecture scales without latency degradation (base-sustained scenario)

The $26\times$ increase in load between the base and sustained scenarios (from ~ 8 to ~ 208 ev/s) results in a mean P95 increase of only 46 ms, i.e. $\sim 4.5\%$ of the base latency. Given the intrinsic variability of each scenario (std of 36.1 ms in the base scenario and 10.3 ms in the sustained scenario), this increase is not statistically significant. The visual comparison between Figure 3 and Figure 4 confirms this stability: the base CDF curves are overlapping (range 975–1,038 ms) and the sustained bars show no increasing trend between repetitions (P95 range 1,054–1,074 ms).

From an operational standpoint, Lambda's automatic scaling (from ≤ 3 concurrent invocations in the base scenario to ≤ 11 in the sustained scenario) absorbs the additional event volume with no operational intervention and no perceptible degradation in response time. Kinesis's IteratorAge remains at 0 throughout the sustained scenario, confirming that the pipeline consumes events as they arrive with no accumulation in the buffer. Overall, the correct-operation envelope of the architecture extends, at least, up to 208 ev/s under sustained load conditions.

8.2.2 The peak scenario reveals the breaking point of the current configuration

The peak scenario reveals two degradation mechanisms operating in cascade, which the current configuration is unable to absorb.

The first mechanism is throttling at the Kinesis ingestion layer. The 5 configured shards provide a theoretical capacity of 5,000 records/s. However, the non-uniform distribution of events by partition key and the instantaneous variations of the generator cause some shards to exceed the individual limit of 1,000 records/s, generating put_records errors. Rejected events never reach Lambda: this explains why the DLQ records 0 messages and the Lambda error rate is 0%, despite an overall loss of 14.195%. The loss breaks down into 12.6% due to throttling at the Kinesis ingestion layer and an additional 2.09% attributable to events that remained in the buffer or in processing at the time the analysis was run.

The second mechanism is inherited backlog between repetitions. Figure 5 shows that the P95 grows almost linearly from second 0 without ever reaching a steady state: the system ingests events faster than it can process them, generating a growing queue in the Kinesis buffer. The 60-second interval between repetitions is insufficient to clear this backlog (~3.4 minutes of recovery time needed), so Run 2 and Run 3 begin with an already-elevated initial P95. Figure 6 confirms this effect: the three distributions show a strong horizontal shift, in contrast to the overlap observed in Figure 3 (base scenario). The P95 grows from 134,184 ms (Run 1) to 197,602 ms (Run 3), an unmistakable signature of inherited accumulation.

In summary, the current configuration (5 shards and parallelization_factor=10) is unable to absorb the load of the peak scenario or to recover within the required time. The causes do not lie in Lambda's processing logic, which operates without errors, but in the Kinesis consumption layer, as argued in Section 8.2.3.

8.2.3 The bottleneck is the Kinesis buffer, not Lambda

A cross-cutting observation across the three scenarios reveals the component that accounts for the entirety of the measured delay: the Kinesis buffer. In all scenarios and percentiles analysed, processing_latency_ms is 0 ms, while kinesis_to_lambda_ms virtually coincides with pipeline_latency_ms (see Appendix 6). This indicates that the entirety of the measured delay accumulates in the Kinesis buffer, while the batch waits to be consumed by Lambda, and not during internal processing.

It should be noted that processing_latency_ms excludes writes to S3 and DynamoDB; nonetheless, the total Lambda invocation duration confirmed by CloudWatch (~277 ms in the sustained scenario and ~2,594 ms in the peak scenario) is far lower than the waiting time in the Kinesis buffer in both scenarios (P50 of 608 ms and 75,394 ms respectively), quantitatively confirming that Lambda is not the bottleneck even when writes are taken into account (see Figure 7, Appendix 5).

Finally, an observation relevant to capacity planning should be highlighted: in the peak scenario, ≤ 11 concurrent

invocations were observed despite the configured maximum of 50 (5 shards $\times$ parallelization_factor=10). The current partition key (subject_id#sensor_type) produces an uneven distribution when sensors with very heterogeneous frequencies coexist: shards assigned to high-frequency sensors (RespiBAN at 700 Hz) receive up to 175 times more events than those of low-frequency sensors (Empatica EDA/TEMP at 4 Hz), leaving some shards underutilised. This observation indicates that Kinesis capacity planning must take into account not only the total event volume but also its distribution across partition keys, especially in scenarios with devices of very heterogeneous frequencies.

8.2.4 Evaluation of requirements and trade-offs

Table 7 presents the evaluation of the system's requirements per scenario.

Table 7. Evaluation of the system's requirements per scenario.

Requirement	Base (~8 ev/s)	Sustained (~208 ev/s)	Peak (~4,304 ev/s)
Latency P95 < 10,000 ms	✓ (1,037 ms)	✓ (1,074 ms)	X (197,602 ms)
Loss < 1%	✓ (0.000%)	✓ (0.000%)	X (14.195%)
Real throughput $\geq$ theoretical	✓ (100%)	✓ (100%)	X (78.3%)
Error tolerance (DLQ = 0)	✓	✓	✓
Lambda scalability	✓ (≤ 3 inv.)	✓ (≤ 11 inv.)	✓ (≤ 11 inv.)
Backlog recovery < 60 s	✓ (0 backlog)	✓ (0 backlog)	X (~3.4 min)

The base and sustained scenarios meet all the requirements in full. The peak scenario, on the other hand, does not meet the latency, integrity or throughput requirements, although error tolerance and Lambda scalability remain satisfied even under extreme conditions. The failures in the peak scenario are not processing failures, but a consequence of the undersizing of the Kinesis ingestion layer.

Regarding the architectural trade-offs from Chapter 5, the results validate them for the most part: the near real-time model shows a margin of more than $9\times$ over the SLO; the serverless model absorbs a $26\times$ load increase without operational intervention; the DLQ was not triggered in any scenario; and DynamoDB responded without degradation in all cases. The single-region vs multi-region trade-off is not assessable in the context of the prototype. Finally, the experiments revealed a trade-off not identified initially: the subject_id#sensor_type partition key guarantees temporal order per sensor but produces an uneven load distribution between shards in the presence of very heterogeneous frequencies, an aspect to be considered in future extensions of the system.

CONCLUSIONS

This work has experimentally demonstrated the viability of the serverless model for the near real-time processing of biomedical data in non-critical hospital areas. The results obtained confirm that the proposed architecture meets the stated objectives and provide a technical diagnosis grounded in experimental evidence regarding its behaviour and its limits under increasing load. Table 8 sets out the degree to which each specific objective was achieved.

Table 8. Degree of achievement of the specific objectives.

Objective	Key result	Status
Identify requirements	5 categories of non-functional requirements with quantifiable SLOs derived from the clinical context	Achieved
Design the architecture	Fully serverless distributed architecture on AWS, with justified decisions and evaluated alternatives	Achieved
Implement the prototype	Functional prototype deployed on AWS via IaC with Terraform; documented public repository	Achieved
Define the experimental methodology	3 scenarios of increasing load, 3 repetitions each, quantitative metrics of latency, integrity and throughput	Achieved
Analyse experimentally	Kinesis buffer identified as the bottleneck; partition-key distribution identified as a critical variable for capacity planning	Achieved

The experimental results validate the architecture for the use case considered. The base and sustained scenarios confirm that the serverless model absorbs load increases of up to 26× with no perceptible latency degradation or event loss. Lambda scales automatically from 3 to 11 invocations with no operational intervention, and Kinesis's IteratorAge remains at zero in both scenarios. The peak scenario, on the other hand, reveals the undersizing of the ingestion layer as the limiting factor: Kinesis throttling causes a loss of 14.195% and a backlog that takes ~3.4 minutes to clear.

One of the most relevant contributions of this work is the experimental diagnosis of the bottleneck: the entirety of the measured delay accumulates in the Kinesis buffer and not in Lambda's internal processing. This distinction is critical for correctly guiding future optimisations, since it implies that increasing Lambda's memory or internal parallelism would not solve the problem.

The experiments also revealed that the coexistence of sensors with very heterogeneous frequencies (from 4 Hz to 700 Hz) affects load distribution between shards and limits Lambda's real parallelism. For this reason, capacity planning must take into account not only the total event volume but also its distribution across partition keys.

From an economic perspective, the operational cost is consistent with the sustainability of the adopted serverless paradigm: the experimental session cost ~\$0.533. In production, the dominant cost would be Amazon Kinesis, with a fixed cost of ~\$2.04/day in PROVISIONED mode, to which the variable cost of DynamoDB and S3 would need to be added, depending on the data volume of the

clinical device adopted.

The main limitations of the study have been documented and quantified throughout the report: the clock skew between the simulator and the AWS domain, the use of synthetic data, and the evaluation of a single infrastructure configuration.

In the short term, priority optimisations act on the Kinesis consumption layer: increasing the shard_count from 5 to 10, increasing Lambda's memory to better exploit the parallelization_factor, and revisiting the partition-key design to balance load distribution in the presence of heterogeneous frequencies. These improvements should be validated through the systematic exploration of intermediate infrastructure configurations, in order to precisely identify the breaking point of the current configuration. In the medium term, the system could be extended with physiological-range validation in the Lambda pipeline, automatic DLQ reprocessing, and an evaluation of Apache Flink for deployments with hundreds of simultaneous patients. In the long term, the projection towards real clinical production requires addressing compliance with privacy regulations (GDPR), NTP synchronisation on the wearable devices, and multi-region deployment for high availability (achievable within minutes thanks to infrastructure as code with Terraform).

This work highlights that in distributed systems, experimental diagnosis is as important as the initial design: detecting silent collisions in DynamoDB, identifying that the bottleneck was the Kinesis buffer and not Lambda, or discovering that frequency heterogeneity between sensors affects load distribution are lessons that only emerge by running real experiments. Furthermore, working with biomedical data, even if synthetic, prompted reflection on the ethical dimension of data engineering: seemingly technical decisions such as the design of partition keys, event traceability, or error handling have direct implications for the integrity of clinical information and, ultimately, for patient safety.

Overall, this work contributes to experimentally grounding the viability of the serverless model for continuous biomedical monitoring applications in non-critical areas, a field in which empirical evidence on architectures of this kind is still limited. The serverless paradigm entails inherent trade-offs, such as reduced control over low-level parallelism, dependency on the cloud provider, and occasional cold starts. In exchange, it offers operational simplicity, automatic scalability and reduced cost, as confirmed by the base and sustained scenarios.

ACKNOWLEDGEMENTS

I would like to especially thank my supervisor, Esteve Alquézar, for his dedication and guidance throughout the project. His technical contributions and constant support have been fundamental to the quality of this work. I would also like to thank my family for their unconditional support throughout the process.

REFERENCES

- [1] C. Downey, W. Tahir, R. Randell, J. Brown, and D. Jayne, "Strengths and limitations of early warning scores: A systematic review and narrative synthesis," *Int. J. Nursing Studies*, vol. 76, pp. 106–119, 2017.
- [2] G. J. Williams et al., "Wearable technology and the cardiovascular system: The future of patient assessment," *The Lancet Digital Health*, 2023.
- [3] A. Keogh et al., "Six-month pilot testing of a digital health tool to support effective self-care in people with heart failure: Mixed methods study," *JMIR Formative Research*, vol. 8, p. e52442, 2023.
- [4] H. Xie, L. Yang, B. Jiang, Z. Huang, and Y. Lin, "State-of-the-art wearable sensors for cardiovascular health: A review," *npj Cardiovascular Health*, vol. 2, no. 53, 2025.
- [5] D. S. Temple et al., "Modular open-core system for collection and near real-time processing of high-resolution data from wearable sensors," *Applied System Innovation*, vol. 6, no. 5, p. 79, 2023.
- [6] A. Bahmani et al., "A scalable, secure, and interoperable platform for deep data-driven health management," *Nature Communications*, vol. 12, p. 5757, 2021.
- [7] A. Orro et al., "A cloud infrastructure for health monitoring in emergency response scenarios," *Sensors*, vol. 24, p. 6992, 2024.
- [8] M. Shafiei, H. Khazaei, and M. Litoiu, "Serverless computing: A survey of opportunities, challenges, and applications," *ACM Computing Surveys*, vol. 54, no. 11s, pp. 1–36, 2022.
- [9] H. B. Hassan, S. A. Barakat, and Q. I. Sarhan, "Survey on serverless computing," *J. Cloud Computing*, vol. 10, no. 1, p. 39, 2021.
- [10] E. Shojaee Rad, M. Babaei, M. Esmaili, and H. Khazaei, "Serverless data processing and analytics: A systematic review," *J. Systems and Software*, vol. 217, p. 112177, 2024.
- [11] G. Wilnai, "Building near-real-time data pipelines," *Datagize*, Feb. 2025. [Online]. Available: <https://datagize.us/building-near-real-time-data-pipelines/>
- [12] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media, 2016.
- [13] UCI Machine Learning Repository, "WESAD: Wearable Stress and Affect Detection," 2018. [Online]. Available: <https://archive.ics.uci.edu/dataset/465/wesad+wearable+stress+and+affect+detection>
- [14] Amazon Web Services, "Amazon Kinesis Data Streams Developer Guide," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/streams/latest/dev/introduction.html>
- [15] Amazon Web Services, "AWS Lambda Developer Guide," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>
- [16] Amazon Web Services, "Amazon Managed Service for Apache Flink," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/managed-flink/latest/java/whatis.html>
- [17] Amazon Web Services, "Amazon S3 User Guide," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>
- [18] Amazon Web Services, "Amazon DynamoDB Developer Guide," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>
- [19] Amazon Web Services, "Amazon Timestream Developer Guide," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/timestream/latest/developerguide/what-is-timestream.html>
- [20] Amazon Web Services, "Amazon CloudWatch User Guide," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html>
- [21] Amazon Web Services, "AWS X-Ray Developer Guide," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/xray/latest/devguide/aws-xray.html>
- [22] Amazon Web Services, "Amazon OpenSearch Service Developer Guide," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/whatis.html>
- [23] Amazon Web Services, "AWS Well-Architected Framework," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>
- [24] Australasian Health Facility Guidelines, "Part B: Health Facility Briefing & Design — Inpatient Unit General," 2012. [Online]. Available: https://www.healthfacilityguidelines.com/View-PDF/ViewIndexPDF/iHFG_part_b_inpatient_unit_general
- [25] Amazon Web Services, "Amazon SQS Developer Guide," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html>
- [26] Amazon Web Services, "Amazon Data Firehose Developer Guide," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/firehose/latest/dev/what-is-this-service.html>
- [27] Apache Software Foundation, "Apache Kafka Documentation," 2024. [Online]. Available: <https://kafka.apache.org/>
- [28] Amazon Web Services, "AWS Glue Developer Guide — Streaming ETL," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/glue/latest/dg/add-job-streaming.html>
- [29] CockroachLabs, "Idempotency and ordering in event-driven systems," CockroachLabs Blog. [Online]. Available: <https://www.cockroachlabs.com/blog/idempotency-and-ordering-in-event-driven-systems/>

STATEMENT ON THE USE OF AI TOOLS AND RESOURCES

For language and spell-checking review of the text: Scribbr. For searching and managing bibliographic references: Anara. The generative AI tools Claude (Anthropic) and ChatGPT (OpenAI) were used as support in reviewing and rephrasing drafting fragments, detecting repetitions, and improving the expository structure. They were not, under any circumstances, used to generate technical content, experimental results, or the conclusions of this work.

APPENDIX

Appendix 1. Dataset and data structure

The WESAD dataset [13] contains multimodal data collected via two wearable devices: the Empatica E4, oriented towards lightweight monitoring, and the RespiBAN, oriented towards high-frequency capture. Table 9 sets out the characteristics of the sensors used as a reference for the simulator design.

Table 9. Characteristics of the WESAD dataset sensors.

Device	Sensor	Physiological signal	Sampling frequency	Data type	Unit	Axes
RespiBAN	ECG	Cardiac activity	700 Hz	Float	mV	1
	EDA	Electrodermal activity	700 Hz	Float	μ S	1
	EMG	Muscular activity	700 Hz	Float	mV	1
	TEMP	Body temperature	700 Hz	Float	$^{\circ}$ C	1
	RESP	Respiration	700 Hz	Float	%	1
	ACC	Acceleration	700 Hz	Float[3]	g	3 (X, Y, Z)
Empatica E4	BVP	Blood volume pulse (PPG)	64 Hz	Float	-	1
	ACC	Acceleration	32 Hz	Float[3]	1/64 g	3 (X,Y, Z)
	EDA	Electrodermal activity	4 Hz	Float	μ S	1
	TEMP	Body temperature	4 Hz	Float	$^{\circ}$ C	1

Table 10 sets out the fields of the raw record, representing the event as generated by the simulator and ingested into the system. Table 11 sets out the additional fields of the processed record, which incorporates the temporal information and latency metrics generated during processing.

Table 10. Fields of the raw record.

Field	Type	Description
event_id	String	Universally unique identifier of the event. Guarantees uniqueness and traceability regardless of temporal resolution.
subject_id	String	Subject identifier.
device_id	String	Device identifier.
sensor_type	String	Sensor type.
sampling_rate_hz	Integer	Sensor sampling frequency.
sensor_timestamp	Float	Sample generation timestamp (Unix UTC).
value	Float / Object	Measurement value. Scalar for single-axis sensors and object for tri-axial sensors.
unit	String	Unit of measurement.
scenario	String	Experimental scenario (base, sustained, peak).
schema_version	String	Data schema version.

Table 11. Additional fields of the processed record.

Field	Type	Description
kinesis_arrival_ts	Float	Timestamp of receipt by Kinesis (Unix UTC). T0 of the temporal model
ingest_timestamp	Float	Timestamp of processing start by Lambda (Unix UTC)
processed_timestamp	Float	Timestamp of processing completion (Unix UTC)
kinesis_to_lambda_ms	Float	Waiting time in the Kinesis buffer (ms)
processing_latency_ms	Float	Internal Lambda processing time (ms)
pipeline_latency_ms	Float	End-to-end latency within the AWS domain (ms). Main SLO metric

Table 12 shows complete examples of the two record types: raw and processed, respectively.

Table 12. Examples of raw and processed records.

Raw record	Processed record
<pre>{ "event_id": "a3f2c1d4-8b7e-4a9f-b2c3-1d4e5f6a7b8c", "subject_id": "S1", "device_id": "EmpaticaE4", "sensor_type": "EDA", "sampling_rate_hz": 4, "sensor_timestamp": 1710000.123, "value": 0.85, "unit": "μS", "scenario": "base", "schema_version": "1.0" }</pre>	<pre>{ "event_id": "a3f2c1d4-8b7e-4a9f-b2c3-1d4e5f6a7b8c", "subject_id": "S1", "device_id": "EmpaticaE4", "sensor_type": "EDA", "sampling_rate_hz": 4, "sensor_timestamp": 1710000.123, "kinesis_arrival_ts": 1710000.587, "ingest_timestamp": 1710001.201, "processed_timestamp": 1710001.203, "value": 0.85, "unit": "μS", }</pre>

	<pre> "scenario": "base", "schema_version": "1.0", "kinesis_to_lambda_ms": 614.0, "processing_latency_ms": 2.0, "pipeline_latency_ms": 616.0 </pre>
--	---

Appendix 2. Technical decisions and alternatives considered

The following technical design decisions are detailed below which, for reasons of length, were not presented in the main body of the report:

- **DynamoDB partition-key design:** using `subject_id` alone as the partition key would concentrate all writes for a given subject in a single partition, potentially exceeding the limit of 1,000 WCU/s per partition in high-frequency scenarios. With the concatenation `subject_id#sensor_type`, the maximum load per partition in the peak scenario (RespiBAN at 700 Hz per sensor) is ~ 700 ev/s, within DynamoDB's limit.
- **UUID as sort key:** in scenarios with multiple repetitions or high sampling frequencies, two events from the same sensor may generate identical timestamps if clock resolution is insufficient. Using a UUID guarantees traceability and write-operation idempotency throughout the pipeline [29].
- **CloudTrail in production:** in the context of the prototype, only management events are enabled, which are free for the first trail. In a production environment with strict auditing requirements, all three event types would be enabled and the log bucket would be configured in Vault mode with WORM (Write Once, Read Many) functionality, which prevents logs from being modified or deleted, even by the root user.

Table 13 sets out the AWS services evaluated and discarded for each architecture component.

Table 13. Alternative AWS services considered and discarded, including the main reason for their exclusion.

Component	Service	Reason for discarding
Ingestion	SQS FIFO	Absence of native reprocessing and insufficient scalability for the peak load (RespiBAN at 4,200 ev/s per subject) [25]
	Amazon Firehose	Minimum latency of 60 s, incompatible with near real-time processing [26]
	Amazon MSK (Kafka)	Absence of a Free Tier and disproportionate operational complexity for a prototype; technically well suited for production with hundreds of simultaneous patients [27]
Processing	Apache Flink	High operational complexity and base cost without a Free Tier; technically well suited for high-load production
	AWS Glue	ETL orientation and latency of seconds to minutes, unsuitable for lightweight per-event operations [28]
Persistence	S3 + Athena	Latency and cost per scan for frequent queries; absence of direct access by key
	Amazon Timestream	Absence of a significant Free Tier; technically superior for biomedical time series [19]
Monitoring	AWS X-Ray	Absence of a native alarm system and inability to compare counts between blocks
	Amazon OpenSearch	Cost of an active cluster per node/hour and excessive complexity for a prototype [22]

Appendix 3. AWS service configuration

Table 14 sets out the complete configuration of the prototype's AWS services.

Table 14. Configuration of the prototype's AWS services.

Component	Parameter	Configuration
Kinesis Data Streams	Shards	5 (PROVISIONED)
	Retention	24 hours
Lambda	Runtime	Python 3.11, 128 MB
Lambda Event Source Mapping	Batch_size	500
	Parallelization_factor	10
	Bisect_batch_on_function_error	Enabled
S3	Raw object key	raw/{timestamp}_{request_id}.json
DynamoDB	Billing mode	PAY_PER_REQUEST
	Write mode	batch_writer
CloudWatch	Frequency	60 seconds
	Retention	15 months
	Namespace	BiomedicalPipeline
AWS Budgets	Alerts	80% and 100% of the monthly \$10 budget

The prototype is deployed in the eu-west-1 region (Europe, Ireland), prioritising service availability and the maturity of the AWS infrastructure. IAM roles follow the principle of least privilege: each system component has a specific role with only the permissions strictly necessary for its function, with no access to resources that do not correspond to it.

Regarding Kinesis, the 5 shards in PROVISIONED mode provide an ingestion capacity of up to 5,000 records/s, and the 24-hour retention guarantees reprocessing in the event of failure. The batch_size of 500 corresponds to the maximum limit of the Kinesis put_records API call; the parallelization_factor of 10 allows up to 50 concurrent invocations (5 shards $\times$ 10); and bisect_batch_on_function_error isolates faulty events by splitting the batch in half. Regarding S3, the object key includes the invocation's request_id to guarantee uniqueness under high parallelism. Regarding DynamoDB, PAY_PER_REQUEST mode adapts to variable load, and batch_writer groups the 500 writes into $\sim$ 20 batch calls. Regarding CloudWatch, custom metrics are published every 60 seconds under the BiomedicalPipeline namespace, with a 15-month retention period. Finally, AWS Budgets was configured with alerts at 80% and 100% of a \$10/month budget to ensure preventive cost control.

Appendix 4. Cost estimate for the experimental session

The estimate for the experimental session was prepared manually based on AWS's public pricing list for the eu-west-1 region (June 2026), since the AWS Pricing Calculator assumes constant monthly load and does not reflect the reality of a $\sim$ 2 h session with highly variable loads. The volumes used correspond to theoretical events, since Kinesis and DynamoDB billing is based on attempted operations, regardless of throttling. The breakdown by service is presented in Table 15.

Table 15. Cost estimate for the experimental session (eu-west-1, June 2026).

Service	Units consumed	Calculation	Estimated cost
AWS Kinesis Data Streams	5 shard-hours + 581,760 events	(5 shards $\times$ 1 h $\times$ \$0.017/shard-hour) + (581,760 PUT Payload Units / 1,000,000 PUT Payload Units $\times$ \$0.0165)	\$0.095
AWS Lambda	581,760 events / 500 batch = $\sim$ 1,164 invocations; $\sim$ 1,164 invocations $\times$ 1 s duration (conservative) $\times$ (128 MB Lambda / 1,024 MB/GB) = 0.125 GB	Covered by the Free Tier: 1M invocations/month + 400,000 GB-s/month	\$0.00
Amazon DynamoDB	581,760 WRU + 58,176 RRU	(581,760 WRU / 1,000,000 WRU $\times$ \$0.705) + (58,176 RRU / 1,000,000 RRU $\times$ \$0.1415)	\$0.418
Amazon S3	581,760 events $\times$ 1,024 bytes + 1,164 PUT requests	(0.596 GB $\times$ \$0.023/GB) + (1,164 PUT requests / 1,000 PUT requests $\times$ \$0.005)	\$0.020
Amazon CloudWatch	4 metrics	Covered by the Free Tier: first 10 metrics/month	\$0.00
Total experimental session			$\sim$\$0.533

As an indicative figure, the dominant cost of a continuous production deployment is Amazon Kinesis, which bills for provisioned capacity regardless of volume. In PROVISIONED mode (5 shards), the fixed cost would be $\sim$ \$2.04/day (5 shards $\times$ 24 h $\times$ \$0.017/shard-hour); in ON-DEMAND mode, billing is per unit processed, which is more economical for variable loads or loads below nominal capacity. Lambda and CloudWatch would remain covered by the Free Tier up to scales far higher than those evaluated. The variable cost of DynamoDB and S3 will depend on the data volume of the clinical device adopted; extrapolating from the experimental session, the total cost is estimated to be in the range of $\sim$ \$2-7/day for a hospital ward in non-critical areas.

Appendix 5. Iterative optimisation process

During the execution of the experiments prior to the final evaluation, four limitations were detected that led to optimisations of the system configuration.

First, the initial single shard (1 shard, limit 1,000 writes/s) fully saturated the stream in the peak scenario ($\sim$ 4,304 ev/s), generating hundreds of "records failed in put_records" warnings. The solution was to progressively increase the shard_count to 5. This configuration provided a nominal capacity of 5,000 writes/s, above the theoretical maximum of the experimental scenarios ($\sim$ 4,303 ev/s).

Second, a loss of 3.944% of records was detected in the base scenario despite an empty DLQ and 0 Lambda errors. The original DynamoDB sort key was sensor_timestamp, the sample generation timestamp from the simulator, which seemed sufficient to uniquely identify each record. However, analysis of the distribution by run_id and sensor_type revealed a symmetric loss between EDA and TEMP that increased across repetitions (26; 49; 67 records lost per run), consistent with silent overwrites: in consecutive repetitions, the simulator generated continuous timestamps using time.time() that could coincide with those of the previous repetition, silently overwriting existing records. The solution was to replace sensor_timestamp with an event_id UUID as the sort key, guaranteeing the absolute uniqueness of each record regardless of temporal resolution.

Third, a growing backlog was detected in the sustained scenario with no obvious cause: the DLQ was empty, there was no Kinesis throttling, and the shard configuration was sufficient. Since processing_latency_ms does not include DynamoDB writes, the cause was not directly observable in the metrics. Analysis of the CloudWatch metrics revealed a mean invocation duration of 7,526 ms, consistent with 500 synchronous put_item calls of $\sim$ 15 ms each. Replacing this with batch_write, which groups the 500 writes into 20 batch calls, reduced the duration to $\sim$ 200 ms and increased

throughput by a factor of $\sim 37\times$, stabilising the P95 within the range set out in Section 8.

Figure 7, on the left, shows the CloudWatch Lambda metrics with individual `put_item`: mean duration of 7,526 ms and maximum of 19,546 ms, consistent with 500 synchronous calls of ~ 15 ms each. On the right, the metrics with `batch_write` are shown: mean duration of ~ 277 ms, a $\sim 27\times$ reduction compared with the previous configuration.

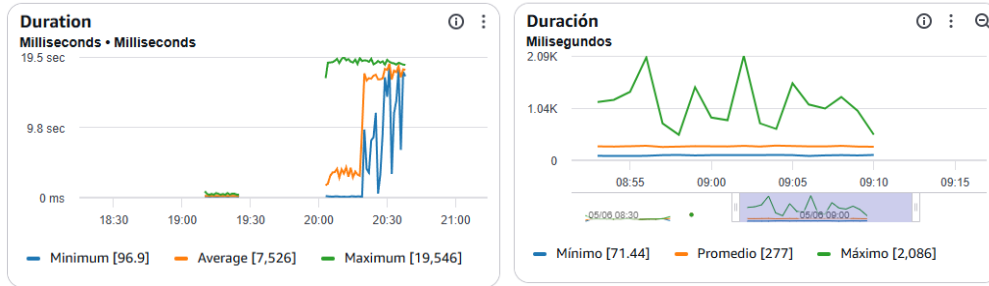


Figure 7. CloudWatch Lambda metrics. Left: individual `put_item` configuration (mean duration 7,526 ms, maximum 19,546 ms). Right: `batch_write` configuration (mean duration 277 ms, maximum 2,086 ms).

Although a metric disaggregated by S3 and DynamoDB separately is not available, the total invocation duration reflected in CloudWatch includes both writes. With `batch_write`, the total duration of ~ 277 ms in the sustained scenario and $\sim 2,594$ ms in the peak scenario confirms that the writes do not represent a bottleneck, since, as set out in Section 8, the waiting time in the Kinesis buffer is far higher in all scenarios.

Finally, it was detected that 46% of records showed negative `pipeline_latency_ms` values (~ -74 ms) due to desynchronisation between the local clock (sensor_timestamp) and the AWS clock. Replacing `time.time()` with Kinesis's `approximateArrivalTimestamp` did not work due to insufficient resolution (1 second). The final decision was to use `kinesis_arrival_ts` as T0 of the temporal model, excluding the generator-to-Kinesis segment from the main metric, as documented in Sections 7.1 and 7.5.

Appendix 6. Breakdown of latencies by component

Table 16. Breakdown of `kinesis_to_lambda_ms` and `processing_latency_ms` per scenario and repetition.

Scenario	run_id	kinesis_to_lambda_ms_p50	kinesis_to_lambda_ms_p95	kinesis_to_lambda_ms_p99	kinesis_to_lambda_ms_mean	processing_latency_ms_p50	processing_latency_ms_p95	processing_latency_ms_p99	processing_latency_ms_mean
Base	1	564	1037	1063	624	0	0	0	0
	2	564	975	1071	537	0	0	0	0
	3	565	1038	1115	584	0	0	0	0
Sustained	1	608	1074	1698	621	0	0	0	0
	2	597	1054	1629	608	0	0	0	0
	3	597	1060	1632	607	0	0	0	0
Peak	1	49911	134184	150538	54888	0	0	0	0
	2	75394	156835	165952	79679	0	0	0	0
	3	97475	197602	205903	104263	0	0	0	0